Architektura Transformer: Tokeny, Okno Kontekstowe i Mechanizm Attention

Moduł 1: Fundamenty i Mechanizmy LLM — Sekcja 1.1

Przed Transformerem: Era RNN i LSTM

Przed rokiem 2017 przetwarzanie języka naturalnego (NLP) opierało się na **rekurencyjnych sieciach neuronowych (RNN)** i ich zaawansowanych wariantach, takich jak **LSTM (Long Short-Term Memory)**.

Modele te analizowały tekst **sekwencyjnie** — słowo po słowie, od lewej do prawej. Podejście to miało dwie poważne wady:

- **Wolne trenowanie** — niemożność zrównoleglenia operacji na dużych zbiorach danych
- **Problem długich zależności** — trudność w utrzymaniu relacji między odległymi fragmentami tekstu (np. podmiot i orzeczenie rozdzielone wieloma zdaniami)
- **Wąskie gardło informacji** — cały kontekst historyczny był kompresowany do jednego wektora stanu ukrytego

Przełom 2017: „Attention Is All You Need"

W 2017 roku zespół badaczy Google opublikował przełomową pracę naukową **„Attention Is All You Need"**, która zaproponowała zupełnie nową architekturę — **Transformer**.

Kluczowa zmiana: porzucenie sekwencyjności na rzecz **przetwarzania równoległego**. Zamiast analizować tekst od lewej do prawej, model może jednocześnie „spojrzeć" na wszystkie tokeny wejściowe i ocenić relacje między nimi.

Transformer stał się fundamentem niemal wszystkich współczesnych dużych modeli językowych (LLM):

- **GPT-4, Claude, Gemini, Llama** — wszystkie opierają się na tej architekturze lub jej bezpośrednich pochodnych
- Zrozumienie Transformera to zrozumienie **silnika napędzającego całą rewolucję AI**

Kluczowe Składniki Architektury Transformer

Architektura Transformer składa się z kilku współpracujących ze sobą modułów:

Składnik	Rola
Embedding	Zamienia tokeny na wektory liczbowe reprezentujące znaczenie
Positional Encoding	Dodaje informację o kolejności tokenów w sekwencji
Multi-Head Attention	Mechanizm uwagi — śledzi relacje między tokenami
Feed-Forward Layers	Warstwy gęste przetwarzające każdy token niezależnie
Layer Normalization	Stabilizuje wartości między warstwami
Residual Connections	Zapobiegają zanikaniu gradientu podczas trenowania

Każdy blok Transformera łączy te elementy, a model składa się z wielu takich bloków ułożonych jeden na drugim.

Tokeny — Jak Model „Widzi" Tekst

Model językowy nie czyta tekstu tak jak człowiek. Zamiast liter lub słów, operuje na **tokenach** — kawałkach tekstu odpowiadających fragmentom słów, całym słowom lub znakom interpunkcyjnym.

Tokenizacja odbywa się **przed** przekazaniem tekstu do modelu, za pomocą algorytmu **BPE (Byte Pair Encoding)** lub jego wariantów (WordPiece, SentencePiece).

Jak działa BPE?

1. Algorytm analizuje częstotliwość par znaków w korpusie treningowym
2. Najczęściej występujące pary są scalane w nowe podjednostki (*subwords*)
3. Powtarzając ten proces, tworzy się słownik podjednostek optymalny dla danego języka
4. Wynikiem jest słownik tokenów, który model wykorzystuje do kodowania dowolnego tekstu

Tokeny — Obserwacje Praktyczne

Kilka ważnych faktów o tokenach, które mają bezpośrednie znaczenie dla pracy z modelami:

- Słowo `„programowanie"` może być jednym tokenem lub rozbite na `„program"` + `„owanie"` — zależy od modelu i jego słownika
- Liczba tokenów **nie jest równa** liczbie słów: angielski tekst zawiera przeciętnie 1 token na ~4 znaki; **języki słowiańskie (w tym polski) są tokenizowane mniej efektywnie** — generują więcej tokenów na słowo
- Każdy token ma swój **indeks numeryczny (ID)** — dla modelu tekst to ciąg liczb całkowitych, a nie liter
- **Koszty API** są naliczane w tokenach, nie w słowach — optymalizacja tokenów w promptach to realna oszczędność przy dużej skali

Narzędzia do liczenia tokenów:

- platform.openai.com/tokenizer — interaktywny tokenizer online
- Pakiet `js-tiktoken` w Node.js: `npm install js-tiktoken`

Okno Kontekstowe — Pamięć Robocza Modelu

Okno kontekstowe (*context window*) to maksymalna liczba tokenów, jaką model może jednocześnie „widzieć" podczas generowania odpowiedzi.

Obejmuje ono **łącznie**:

- Prompt (polecenie, instrukcje systemowe)
- Historię rozmowy
- Dołączone dokumenty lub fragmenty kodu
- Generowaną odpowiedź

Analogia: Wyobraź sobie kartkę papieru, na której możesz zapisać tylko określoną liczbę linii. Czytając długi tekst, musisz przewijać — wcześniejsze fragmenty „wypadają" z pola widzenia i przestajesz móc się do nich odwoływać. Okno kontekstowe działa dokładnie tak samo.

Okno Kontekstowe — Porównanie Modeli

Różne modele mają bardzo różne rozmiary okna kontekstowego:

Model	Okno kontekstowe	Przybliżony ekwiwalent
GPT-3.5 Turbo	16 384 tokenów	~12 tys. słów
GPT-4o	128 000 tokenów	~96 tys. słów
Claude 3.5 Sonnet	200 000 tokenów	~150 tys. słów
Gemini 1.5 Pro	1 000 000 tokenów	~750 tys. słów
Llama 3 (8B)	8 192 tokenów	~6 tys. słów

Rozmiar okna kontekstowego to jeden z kluczowych parametrów przy wyborze modelu dla konkretnego zastosowania — szczególnie gdy pracujemy z długimi dokumentami lub wieloturową rozmową.

Okno Kontekstowe — Praktyczne Implikacje

Zrozumienie okna kontekstowego jest niezbędne przy projektowaniu aplikacji AI:

1. **Długie dokumenty** — kod całego projektu, transkrypcje spotkań, raporty PDF muszą zmieścić się w oknie kontekstowym lub wymagają strategii podziału (*chunking*) stosowanej w architekturze RAG
2. **Historia rozmowy** — każda kolejna wiadomość zajmuje miejsce; przy bardzo długich sesjach wcześniejsze wiadomości mogą zostać „zapomniane"
3. **Koszty** — przetwarzanie większego kontekstu jest droższe, ponieważ każdy token wejściowy jest płatny
4. **Zjawisko „Lost in the Middle"** — badania pokazują, że modele gorzej „pamiętają" informacje umieszczone w **środku** długiego kontekstu niż na początku lub końcu — kluczowe informacje warto umieszczać na początku lub końcu promptu

Mechanizm Attention — Intuicja

Attention (uwaga) to kluczowa innowacja architektury Transformer. Odpowiada na pytanie:

> *„W kontekście tego tokenu, na które inne tokeny powinienem zwrócić uwagę i z jaką siłą?"*

Intuicja na przykładzie: Rozważ zdanie:

> *„Bank odmówił pożyczki, bo zbankrutował."*

Słowo „zbankrutował" opisuje podmiot — ale który? Człowiek bez trudu powiąże ten czasownik z „bankiem", a nie z „pożyczką". Mechanizm Attention robi dokładnie to samo: dla każdego tokenu oblicza dynamicznie, jak silnie jest powiązany z każdym innym tokenem w kontekście.

Kluczowa różnica w stosunku do RNN: Attention pozwala modelowi „spojrzeć" na odległy token bezpośrednio, bez pośrednich kroków — bez utraty informacji na długich dystansach.

Mechanizm Attention — Query, Key, Value

Mechanizm Attention opiera się na transformacji każdego tokenu w trzy wektory:

Wektor	Pytanie które zadaje	Rola
Query (Q)	„Czego szukam?"	Reprezentuje bieżące zapytanie tokenu
Key (K)	„Co oferuję?"	Reprezentuje, jak każdy token „opisuje siebie"
Value (V)	„Co przekazuję, jeśli zostanę wybrany?"	Treść informacyjna tokenu

Proces obliczeń:

1. Oblicz podobieństwo Q bieżącego tokenu z K wszystkich tokenów (iloczyn skalarny)
2. Podziel przez $\sqrt{d_k}$ (normalizacja — zapobiega eksplodującym gradientom)
3. Zastosuj funkcję **softmax** — zamień podobieństwa na wagi sumujące się do 1
4. Oblicz ważoną sumę wektorów V — to wynik uwagi dla danego tokenu

Mechanizm Attention — Formuła Matematyczna

Formalna definicja Scaled Dot-Product Attention:

$\text{Attention}(Q, K, V) = \text{softmax}!\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Rozumienie intuicyjne formuły:

- QK^T — macierz podobieństw między wszystkimi parami tokenów (każdy z każdym)
- $\sqrt{d_k}$ — czynnik skalujący, gdzie d_k to wymiarowość wektora klucza
- $\text{softmax}(\cdot)$ — normalizuje podobieństwa do rozkładu prawdopodobieństwa (wag)
- Wynik: ważona kombinacja wartości V — każdy token otrzymuje nową reprezentację uwzględniającą kontekst

Ważna praktyczna uwaga: Znajomość formuły nie jest wymagana do pracy z LLM — kluczowe jest rozumienie konceptu: model dynamicznie decyduje, które tokeny są istotne dla danego tokenu w danym kontekście.

Multi-Head Attention — Wiele Perspektyw Jednocześnie

Transformer nie stosuje jednego mechanizmu Attention, lecz **wielogłowicowy Attention** (Multi-Head Attention, MHA). Model równolegle uczy się wielu różnych „głowic uwagi", z których każda specjalizuje się w innym rodzaju relacji między tokenami.

Analogia panelu ekspertów: Wyobraź sobie, że ten sam tekst czyta jednocześnie kilku specjalistów z różnych dziedzin:

- **Głowica 1** — wyłapuje relacje gramatyczne (podmiot–orzeczenie, dopełnienie)
- **Głowica 2** — śledzi koreferencje zaimków (który „on" do kogo się odnosi)
- **Głowica 3** — rozpoznaje relacje semantyczne (synonimy, antonimy, hiponimy)
- **Głowica 4** — wykrywa relacje czasowe (kolejność zdarzeń, związki przyczynowe)

Wyniki wszystkich głowic są łączone i transformowane, tworząc bogatą, wielowymiarową reprezentację znaczenia każdego tokenu.

Multi-Head Attention — Skala w Dużych Modelach

Wielogłowicowy Attention w praktyce przemysłowej osiąga imponującą skalę:

- **GPT-4** — korzysta z kilkudziesięciu głowic uwagi w każdej z wielu warstw
- Każda warstwa dodaje kolejny poziom abstrakcji: niższe warstwy uczą się prostych wzorców składniowych, wyższe — złożonych relacji semantycznych
- Głębokość modelu (liczba warstw) i szerokość (liczba głowic, wymiarowość wektorów) to kluczowe parametry architektury

Praktyczna konsekwencja dla promptowania:

Mechanizm Attention sprawia, że modele doskonale rozumieją **kontekst** — ale wyłącznie w granicach okna kontekstowego. Informacje spoza kontekstu nie istnieją dla modelu podczas generowania. Dlatego właśnie umieszczanie kluczowych informacji w odpowiednim miejscu promptu ma tak duże znaczenie dla jakości odpowiedzi.

Encoder, Decoder i Architektury Hybrydowe

Oryginalna architektura Transformer składała się z dwóch bloków. W świecie LLM wyewoluowały trzy rodziny modeli:

Architektura	Przykłady	Główne zastosowanie
Encoder-only	BERT, RoBERTa	Klasyfikacja tekstu, ekstrakcja informacji, NER
Decoder-only	GPT-4, Claude, Llama	Generowanie tekstu, chatboty, asystenci kodowania
Encoder-Decoder	T5, BART	Tłumaczenie maszynowe, podsumowania

Modele Decoder-only — jak działają?

Praktycznie wszystkie modele, z którymi pracuje AI developer (GPT-4o, Claude, Llama 3), to architektura **decoder-only**. Generują tekst **autoregresywnie** — token po tokenie, za każdym razem na podstawie całego dotychczasowego kontekstu (prompt + już wygenerowane tokeny).

Trenowanie jest równoległe, ale generowanie jest z natury sekwencyjne — model musi najpierw wygenerować token N, zanim może wygenerować token N+1.

Kluczowe Wnioski

Zrozumienie architektury Transformer to fundament świadomej pracy z modelami AI:

- **LLM to statystyczne maszyny** przewidujące następny token na podstawie wszystkich poprzednich tokenów w kontekście
- **„Myślenie" modelu** to wielowarstwowe przekształcenie reprezentacji wektorowych przez mechanizm uwagi — nie ma tu żadnej symbolicznej logiki
- **Okno kontekstowe** to twarda granica — model nie pamięta nic spoza niego; zarządzanie kontekstem to kluczowa umiejętność AI developera
- **Tokeny, nie słowa** — zrozumienie tokenizacji pomaga optymalizować koszty i przewidywać zachowanie modelu
- **Attention = kontekst** — model doskonale rozumie relacje między tokenami w oknie, ale nic poza nim

Świadome zarządzanie kontekstem, tokenami i wyborem modelu to praktyczne umiejętności wynikające bezpośrednio z rozumienia tej architektury.